

Evaluación Parcial 1: Línea base de calidad de tu proyecto

ASIGNATURA	PRO402 · Taller de Testing y Calidad de Software
UNIDAD	01 · Calidad y Testing de Software
MODALIDAD	Práctica individual · Proyecto incremental
DOCENTE	Diego Obando

1 Sentido de la evaluación

Esta es la primera de las tres entregas del proyecto que acompaña todo el módulo. No se evalúa la aplicación: se evalúa **la calidad de la verificación que construiste sobre ella**.

Esa distinción cambia por completo el criterio con que conviene trabajar. Un sistema con tres reglas de negocio bien interrogadas obtiene mejor evaluación que uno con veinte funcionalidades sin evidencia de ninguna. Agregar código no sube la nota. Agregar evidencia sí.

Lo que se te pide demostrar es que tu proyecto tiene una línea base: que se instala igual en cualquier máquina, que pasa sus controles estáticos, que tiene pruebas que describen lo que hoy hace, que sabes contra qué estándar se mide su calidad, y que ya encontraste al menos un problema que ninguna de esas herramientas podía encontrar por ti.

2 El proyecto sobre el que se trabaja

El proyecto nace aquí y te acompaña hasta el final del módulo. Puede ser una idea nueva o la continuación de algo que ya hayas construido; esa decisión es tuya.

Condiciones del alcance:

- El sistema debe estar escrito en Python y tener **entre tres y seis reglas de negocio propias**: decisiones que el sistema toma y que podrían estar mal.

- Una regla de negocio no es una operación de librería. `sumar(a, b)` no es una regla; «un pedido bajo el mínimo no genera despacho», «una nota se aproxima al decimal superior a partir de 0,05» o «un usuario suspendido no puede reservar» sí lo son.
- No se exige interfaz gráfica, base de datos ni API. Si tu proyecto ya las tiene, no las quites, pero tampoco son lo que se evalúa en esta entrega.
- El sistema debe poder ejecutarse. Un proyecto que no corre no es evaluable.

Si tu proyecto tiene menos de tres reglas de negocio, no alcanza para sostener la evaluación. Si tiene muchas más, vas a repartir el esfuerzo en superficie en vez de profundidad, que es exactamente lo contrario de lo que se busca.

3 El entorno de trabajo

El módulo usa un stack fijo, y es **el mismo para las tres evaluaciones**. Lo que montes ahora es lo que vas a seguir usando hasta el final: las entregas siguientes agregan herramientas encima, no reemplazan estas.

Desde esta entrega:

Herramienta	Para qué
<code>uv</code>	Gestiona el proyecto: dependencias, entorno virtual y versión de Python
<code>ruff</code>	Análisis estático del código
<code>pyrefly</code>	Verificación de tipos
<code>pytest</code>	Pruebas automatizadas

Sobre este mismo proyecto se suman después FastAPI para la interfaz consumible, Playwright para las pruebas de extremo a extremo y GitHub Actions para la integración continua. No hace falta instalarlas ahora; sí conviene saber que van a montarse encima de lo que construyas en esta entrega.

Condiciones:

- Las cuatro herramientas se declaran como dependencias de desarrollo del proyecto, no se instalan de forma global. Quien clone el repositorio debe poder obtenerlas con un solo comando.
- La versión de Python queda fijada en `.python-version` y versionada en el repositorio.

- Esa versión se mantiene durante todo el módulo. Si tuvieras que cambiarla, la razón queda registrada en `CALIDAD.md`.
- El stack no se reemplaza: no `pip` ni `poetry` en lugar de `uv`, no `mypy` en lugar de `pyrefly`, no `unittest` en lugar de `pytest`. Parte de lo que se evalúa es el manejo de estas herramientas concretas.

4

Requisitos mínimos para que la entrega sea evaluable

A. Repositorio ejecutable y reproducible

- El código debe estar en GitHub, público o compartido con el docente.
- El entorno se gestiona con `uv`: deben estar versionados el `pyproject.toml`, el archivo de bloqueo y la versión de Python fijada.
- El `README.md` debe permitir que otra persona instale y ejecute el proyecto **y sus controles** siguiendo los pasos, sin preguntarte nada.
- No deben subirse claves, tokens ni contraseñas. Si el proyecto usa variables de entorno, incluye un `.env.example`.

B. Análisis estático en verde

- `pyrefly` debe terminar sin errores con la verificación de tipos activada.
- `ruff` debe terminar sin errores.
- Cada diagnóstico silenciado —un `ignore`, un `noqa`, una regla desactivada— debe estar **justificado por escrito** en `CALIDAD.md`, indicando qué decía el diagnóstico y por qué en ese caso concreto no corresponde. Silenciar sin justificar cuenta como no haberlo resuelto.

C. Suite de pruebas que documenta el comportamiento actual

- La suite se ejecuta con `pytest` y termina en verde.
- Debe haber **al menos una prueba por cada regla de negocio declarada**, y cada prueba debe poder fallar: si cambias la regla que verifica, la prueba tiene que ponerse roja.
- Al menos una prueba debe **reproducir un defecto real** que encuentre en tu propio código, acompañada de la corrección que la puso en verde.
- Las pruebas describen el comportamiento que el sistema tiene hoy, no el que te gustaría que tuviera. Si una regla está mal implementada y decides no corregirla todavía, la prueba documenta la conducta actual y el desacuerdo queda registrado en `CALIDAD.md` como hallazgo.

No hay meta de cobertura y no se pide medirla. Un porcentaje alto se consigue ejecutando líneas sin verificar nada, y por eso no distingue una suite buena de una inútil. Lo que se mira es si las pruebas se caen cuando el comportamiento cambia.

D. Documentación de calidad: `CALIDAD.md`

Un solo archivo en la raíz del repositorio, con cuatro partes:

1. **Ficha de calidad ISO/IEC 25010.** Las tres características priorizadas para tu producto, por qué esas y no otras, y para cada una un criterio redactado de forma verificable.
2. **Tabla de trazabilidad.** Una fila por criterio, con cuatro columnas: necesidad → criterio → evidencia de verificación → evidencia de validación. Las celdas sin respaldo se dejan explícitamente vacías y marcadas como vacías. Una tabla completa e inventada vale menos que una tabla honesta con huecos.
3. **Justificación de los diagnósticos silenciados**, según el punto B.
4. **Hallazgos de la auditoría.** Al menos tres, y **uno de ellos debe ser un hallazgo de validación**: un caso en que tu sistema aprueba todas sus pruebas y aun así entrega un resultado que incumple la regla que debía respetar. Para ese hallazgo hay que explicar por qué es un problema de validación y no un defecto de implementación.

E. Registro del trabajo con agentes

En el `README.md`, una sección `Uso de IA o agentes` con: qué herramientas usaste, para qué, qué revisaste o corregiste tú, y qué error o límite detectaste en lo que te propuso el agente.

Si aplicaste revisión adversarial —un agente produce, otro audita, tú arbitras—, deja registrado al menos un caso completo: qué propuso el primero, qué objetó el segundo y qué decidiste tú.

5 Entregables

1. Enlace al repositorio de GitHub.
2. `README.md` con descripción del proyecto, reglas de negocio declaradas, instrucciones de instalación, comandos para ejecutar los tres controles (`pyrefly`, `ruff`, `pytest`) y la sección `Uso de IA o agentes`.
3. `CALIDAD.md` con sus cuatro partes.
4. Código fuente y suite de pruebas.
5. Historial de commits que permita ver cómo avanzó el trabajo.

6 Qué se evalúa y cuánto pesa

Criterio	Peso
Suite de pruebas que documenta el comportamiento — cobertura real de las reglas declaradas, capacidad de las pruebas de detectar un cambio de conducta, y el defecto reproducido	25%
Evidencia estática — <code>pyrefly</code> y <code>ruff</code> en verde, y calidad de la justificación de cada diagnóstico silenciado	20%
Trazabilidad y ficha de calidad — pertinencia de las tres características ISO/IEC 25010, criterios verificables y honestidad de la tabla necesidad-criterio-evidencia	20%
Hallazgo de validación — profundidad del caso encontrado y solidez del argumento sobre por qué es validación y no implementación	15%
Entorno reproducible y orden de entrega — el proyecto se instala y corre siguiendo el README, sin secretos y con historial útil	10%
Criterio frente al agente — qué delegaste, qué revisaste, qué corregiste y qué decidiste tú	10%

7 Verificación en vivo

En la sesión de evaluación, con tu proyecto ya entregado, se hace lo siguiente frente a ti:

1. Se clona tu repositorio en el estado en que lo entregaste.
2. Se introduce **un cambio pequeño en una de tus reglas de negocio**: un `>=` que pasa a `>`, un redondeo alterado, un límite corrido en uno. Un cambio que un usuario notaría.
3. Se ejecuta tu suite.

Lo que se observa: si alguna prueba se cae, cuál, y si el mensaje de fallo permite entender qué se rompió. Si nada se cae, se te pregunta por qué, y tu explicación forma parte de la evaluación.

Esta verificación alimenta el criterio de suite de pruebas.

Cómo se elige el cambio

El cambio no lo escribe el docente a mano. Lo genera un **agente de IA** —Claude Code o Codex, el mismo para toda la cohorte— sobre el clon de tu repositorio, con estas reglas:

- **Solo toca reglas de negocio que tú declaraste** en tu `README.md`. No se modifica código de soporte, ni configuración, ni pruebas.

- **Sale de un catálogo fijo**, el mismo para todos: invertir un operador de comparación, correr un límite en uno, cambiar el sentido de un redondeo, alterar un valor por defecto o negar una condición. Nada más elaborado que eso.
- **Es un solo cambio**, en un solo lugar.

Antes de ejecutar la suite se te muestra el cambio exacto, y queda registrado junto con el resultado. Como sale de tu propio `README.md` y de una lista cerrada, es el mismo procedimiento para todos aunque el código de cada uno sea distinto.

Puedes impugnar el cambio

Tienes derecho a rechazar el cambio introducido, en el momento, por dos causales:

1. **No altera el comportamiento observable del sistema.** En pruebas de mutación esto se llama un *mutante equivalente*: el código quedó distinto pero hace exactamente lo mismo, así que ninguna prueba podría detectarlo y no prueba nada sobre tu suite.
2. **No corresponde a una regla de negocio que declaraste**, o toca código que no es tuyo.

Si la objeción es correcta, ese cambio se descarta y el agente genera otro. La objeción hay que fundamentarla ahí mismo, mostrando por qué el comportamiento no cambia o dónde está el límite de lo que declaraste.

8

Qué baja la evaluación

- Entregar un proyecto que no ejecuta, o que no se puede instalar siguiendo el README.
- Pruebas que no pueden fallar: sin aserciones, con `assert True`, o que verifican algo que no depende del código propio.
- Silenciar diagnósticos de `pyrefly` o `ruff` de forma masiva, o con justificaciones genéricas copiadas entre casos distintos.
- Desactivar reglas para llegar al verde en vez de resolver lo que señalan.
- Una tabla de trazabilidad completa donde las evidencias no existen en el repositorio.
- Un hallazgo de validación que en realidad es un defecto de implementación, presentado como si fuera lo otro.
- No poder explicar una prueba, una configuración o una decisión que está en tu propia entrega.
- Subir claves, tokens o contraseñas.

Está permitido usar Claude, Codex, ChatGPT, Gemini u otros agentes durante todo el desarrollo. El módulo asume que vas a trabajar así, porque así se trabaja hoy.

Lo que no cambia es de quién es la responsabilidad. En esta evaluación eso se traduce en una regla concreta:

Si un agente escribió una prueba, tienes que poder decir **qué defecto atraparía esa prueba**. Si no lo sabes, la prueba no cuenta como evidencia, aunque esté en verde.

Lo mismo aplica a una corrección de tipos, a una regla de `ruff` silenciada o a un criterio de calidad propuesto. El agente puede proponer; la decisión de aceptar, corregir o descartar es tuya, y es lo que se evalúa.

1. Sube el proyecto a GitHub.
2. Verifica que el repositorio sea público o esté compartido con el docente.
3. Confirma que el `README.md` permite instalar y ejecutar el proyecto y sus tres controles.
4. Envía el enlace por el canal oficial de la asignatura.

El proyecto se construye durante las sesiones, así que conviene que el repositorio esté publicado desde el principio y que los commits acompañen el trabajo. La verificación en vivo se ejecuta sobre el estado que tenga el repositorio en ese momento.